

Department of Computer and Software Engineering

SE: Machine Learning

Course Instructor: Dr. Ahmed Raza

Date: 12-February-2026

Day: Thursday

Semester: 6th

Lab 1: The Engine of Learning: Gradient Descent and KNN

Lab Title	CLO	BT	PLO	Weightage
The Engine of Learning: Gradient Descent and KNN				

Name	Roll Number	Report Marks/10	Viva Marks/5	Total Marks/15
Muhammad Abdul Qadir	BSSE23105			

Checked on: _____

Signature: _____

1.1 Learning Outcomes

After completing this Lab, the student will be able to:

- Implement gradient descent for scalar and vector-valued functions
- Understand convergence, divergence, and learning rate effects
- Implement K-Nearest Neighbors using distance geometry
- Compare loop-based vs vectorized implementations

1.2 Equipment

- PC with Python 3.x installed (Anaconda Distribution recommended).
- IDE of choice (Jupyter Notebook, Spyder, VS Code, or PyCharm).
- NumPy, Pandas, and Matplotlib libraries installed.

1.3 Theory & Background

Machine Learning heavily relies on optimization algorithms and geometric reasoning. In this lab, you will build machine learning algorithms (Gradient Descent and K-Nearest Neighbors) from scratch using math and NumPy. No sklearn. Only optimization, geometry, and engineering discipline.

1.3.1 Gradient Descent

Gradient Descent is an iterative optimization algorithm used to minimize a function by moving in the direction of steepest descent. The update rule is:

$$\theta_{n+1} = \theta_n - \alpha \nabla f(\theta_n)$$

where α is the learning rate and ∇f represents the gradient. The learning rate controls the step size and critically affects convergence behavior.

1.3.2 K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based learning algorithm. It classifies a data point based on the majority class among its k nearest neighbors in the feature space. The algorithm relies on distance metrics, typically Euclidean distance:

$$d(x, y) = \sqrt{\sum_{i=1}^N (x_i - y_i)^2}$$

KNN has no explicit training phase; all computation happens at prediction time, making it memory-intensive for large datasets.

1.4 Submission Format

- Submit the provided filled in python script (.py), along with a manual in PDF format in a zip file.
- Naming convention: BSSE23XXX_SECTION#_ML_LABY (e.g., BSSE23001_A_ML_LAB1), where XXX is the remaining roll number and Y is the lab number.

PART A: Gradient Descent

Task A1: Scalar Gradient Descent [2 Marks]

Function: $f(x) = x^2 + 5$

1. Derive $f'(x)$ analytically.
2. Implement gradient descent starting from $x_0 = 10$.
3. Run for 20 iterations with learning rates: $\alpha = 0.1, 1.0, 1.1$
4. Identify which values converge or diverge and explain why.

Task A2: Gradient Descent on a 2D Surface [2 Marks]

Function: $f(x, y) = x^2 + 3y^2$

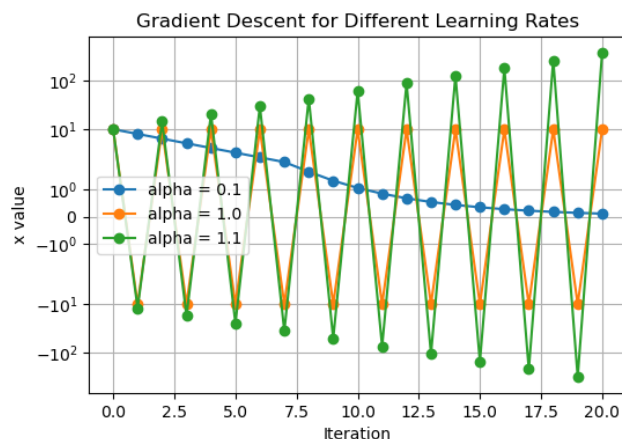
1. Derive the gradient vector $\nabla f = [\partial f / \partial x, \partial f / \partial y]$.
2. Start from $(x_0, y_0) = (5, 5)$ with learning rate $\alpha = 0.1$.
3. Track parameter updates for at least 10 iterations.
4. Explain why convergence rate differs for x and y dimensions.

Task A3: Failure Mode [1 Mark]

1. Choose a learning rate that causes divergence.
2. Log the exploding values over iterations.
3. Explain divergence mathematically: why does the value grow instead of decrease?

Task A1:

- $f'(x) = 2x$



Output: (after 20 iterations)

```
--- Task A1 ---
Alpha 0.1: Final x = 0.1153
Alpha 1.0: Final x = 10.0000
Alpha 1.1: Final x = 383.3760
```

Which value converge or diverge and explanation

- $\alpha = 0.1$ (converges): The steps are small, so each update moves you closer to the minimum smoothly. That's why x keeps shrinking and ends up near 0 (0.1153).
- $\alpha = 1.0$ (oscillates): The step is too “perfectly big,” so it jumps from one side of 0 to the other with the same distance. So it keeps bouncing between +10 and -10 and never settles.
- $\alpha = 1.1$ (diverges): The step is too large, so it overshoots more and more every time. Instead of getting closer to 0, the values grow bigger (like 383.376), so it blows up.

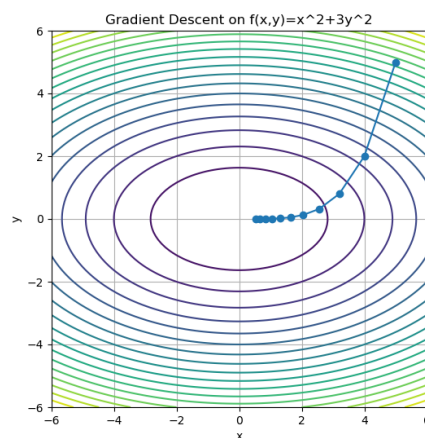
Task A2:

- $\nabla f(x,y)=[2x,6y]$

-----Output -----

```
--- Task A2 ---
Iteration    x        y
0           5.000    5.0000
1           4.000    2.0000
2           3.200    0.8000
3           2.560    0.3200
4           2.048    0.1280
5           1.638    0.0512
6           1.311    0.0205
7           1.049    0.0082
8           0.839    0.0033
9           0.671    0.0013
10          0.537    0.0005
```

Start: [5. 5.] and End: [5.36870912e-01 5.24288000e-04]



Why convergence rate differs for x and y dimensions.?

Because curvature is different.

$$H = \begin{bmatrix} 2 & 0 \\ 0 & 6 \end{bmatrix}$$

Eigenvalues: 2 and 6

Update factors:

- x shrinks by 0.8
- y shrinks by 0.4

Since $0.4 < 0.8$, y goes to zero much faster.

Task A3:

Output

```
--- Task A3 ---
Used 1.1 as learning rate for divergence as in first part it cause divergence
Exploding Values
t = 0    x0 = 10.0000
t = 1    x1 = -12.0000
t = 2    x2 = 14.4000
t = 3    x3 = -17.2800
t = 4    x4 = 20.7360
t = 5    x5 = -24.8832
```

Divergence mathematically

Gradient descent step:

$$x_{t+1} = x_t - \alpha(2x_t) = x_t(1 - 2\alpha)$$

So after iterations:

$$x_t = x_0(1 - 2\alpha)^t$$

Convergence requires:

$$|1 - 2\alpha| < 1$$

But with your learning rate $\alpha=1.1$:

$$1 - 2\alpha = -1.2$$

$$|-1.2| = 1.2 > 1$$

Since the multiplier's magnitude is greater than 1, each iteration increases .

Therefore the values grow exponentially (and alternate sign), so the method diverges.

PART B: K-Nearest Neighbors (KNN)

Dataset: A synthetic 2D binary classification dataset generated using NumPy (please refer to function `generate_dataset` in the code file provided).

The dataset is linearly inseparable and reproducible via a fixed random seed.

Task B1: Euclidean Distance [1 Mark]

1. Implement Euclidean distance between two points using loops.
2. Implement Euclidean distance using NumPy vectorization (no loops).
3. Verify both implementations produce the same result.
4. Compare code clarity and efficiency.

Task B2: Distance Computation with Broadcasting [2 Marks]

1. Compute distances from a single test point to all training points.
2. No loops allowed, use NumPy broadcasting.
3. Explain the broadcasting expression in comments.

Task B3: Neighbor Selection (No Full Sorting) [1 Mark]

1. Use `np.argpartition` to find k nearest neighbors without full sorting.
2. Explain why full sorting with `np.argsort` is inefficient for large datasets.
3. Compare time complexity: $O(n \log n)$ vs $O(n)$.

Task B4: KNN Prediction [2 Marks]

1. Retrieve the labels of the k nearest neighbors.

2. Perform majority voting to determine the predicted class.
3. Return the predicted class label.

Task B5 — Effect of k [1 Mark]

1. Test your KNN implementation with $k = 1$, $k = 5$, and $k = n$ (total training points).
2. Identify which values lead to overfitting and which lead to under fitting.
3. Explain the behavior observed.

Task B1:

Output:

```
Task B1: Euclidean Distance
B1.1-> Loop distance: 1.6681401448799071
B1.2-> Vectorized distance: 1.6681401448799071
B1.3-> Same result: True
B1.4-> Loop time: 0.651476s
B1.4-> Vectorized time: 0.009041s
B1.4-> Speedup: 72.06x
B1.4-> Clarity: Loop is step-by-step; vectorized is shorter and usually faster.
```

Task B2:

```
#B2.2+ B2.3
# training_features shape: (n_train, 2) -> n_train points, each point has 2 values (x, y)
# single_test_point shape: (2,) -> one point with 2 values [x, y]
# training_features -> single_test_point broadcasts the (2,) point across all rows (no Python loop)
# this produces (n_train, 2) diff vectors: diff[i] = training_features[i] - single_test_point
# np.linalg.norm(..., axis=1) converts each diff row into one distance, giving (n_train,) distances
```

Output:

```
Task B2: Distance Computation with Broadcasting
B2.1-> Test point: [-0.5297602  0.51326743]
B2.1-> Distances shape: (70,)
B2.1-> First 10 distances: [4.30110331 0.91361317 4.72338335 2.58088847 0.6707424 1.05260011
1.54220812 1.61576778 3.18782587 1.90894226]
```

Task B3:

Output:

```
Task B3: Neighbor Selection (No Full Sorting)
B3.1-> k: 10
B3.1-> Nearest indices (argpartition, unsorted): [18 27 41 58  4 37 10 33 62 22]
B3.2-> Why not argsort: because it sorts every distance from smallest to largest,
      even though you will only use the first k after sorting. That extra work becomes wasteful when n is large
B3.3-> Complexity: argsort=  $O(n \log n)$ , selection/partition=  $O(n)$  for top-k.
```

Task B4:

Output

```
Task B4: KNN Prediction
B4.1-> Neighbor labels:
Neighbor 1 class = 0
Neighbor 2 class = 0
Neighbor 3 class = 0
Neighbor 4 class = 0
Neighbor 5 class = 0
Neighbor 6 class = 0
Neighbor 7 class = 0
Neighbor 8 class = 0
Neighbor 9 class = 0
Neighbor 10 class = 0
B4.2-> Predicted class (majority vote): 0
Train class counts: class 0 = 34 , class 1 = 36
```

Task B5:

```
Task B5: Effect of k
B5.1-> Accuracy k=1: 0.900
B5.1-> Accuracy k=5: 1.000
B5.1-> Accuracy k=n_train(70): 0.467

B5.2-> Overfitting happens at very small k (like k=1).
B5.2-> Underfitting happens at very large k (like k=n_train=70).

B5.3-> Why: Small k uses only a few closest points, so it can follow noise/outliers and become unstable.
B5.3-> Medium k (like 5) averages nearby votes, so it becomes more stable and often improves accuracy.
B5.3-> When k=n_train, every test point votes using ALL training points,
      so the model predicts the same 'global majority class' (1, count=36) for almost every test point.
      On roughly balanced data, that behaves close to guessing, so accuracy can drop toward almost 50%.
B5.3-> Why does k=101 fail? □
```


PART C — Reflection [3 Marks]

1. Compare Gradient Descent vs KNN in terms of:
 - Training cost
 - Prediction cost
 - Memory usage
2. Which algorithm scales better to large datasets? Justify your answer.
3. Why is KNN rarely used in production ML systems despite its simplicity?

1: Comparison-> Gradient Descent vs KNN

Training cost

- Gradient Descent: Training takes time because it repeatedly updates model parameters using data.
- KNN: Almost no training time, it just stores the training data.

Prediction cost

- Gradient Descent: Prediction is fast because it uses a learned formula.
- KNN: Prediction is slow because it compares the new point with many training points.

Memory usage

- Gradient Descent: Uses little memory (stores only model parameters).
- KNN: Uses a lot of memory (stores the whole dataset).

2: Which scales better to large datasets?

Gradient Descent scales better.

Because once training is done, it makes predictions quickly and does not need to store all training data.

KNN becomes slow and memory-heavy when the dataset is large.

3: Why KNN is rarely used in production?

Even though KNN is simple:

- Predictions are slow for big datasets
- It needs a lot of memory
- Performance drops with many features
- Hard to use efficiently at large scale

So most real systems prefer trained models (like those learned with Gradient Descent) because they are faster and more efficient.

Conclusion [1 Mark]

Write a brief summary of what you learned from this lab, focusing on the importance of understanding algorithmic foundations before using high-level libraries.

This lab made me realize I shouldn't just rely on libraries and copy-paste models. When I coded gradient descent myself, I actually saw how the learning rate can make things converge, get stuck, or blow up. Building KNN from scratch also showed me how distance and the value of k can completely change the results (overfitting vs underfitting). Overall, doing it manually helped me understand what the algorithms are really doing, so I can debug better and make smarter choices later even if I use high-level libraries.

Assessment Rubrics

SE306L: Machine Learning Lab

Method: Lab reports and instructor observation during lab sessions

Outcome Assessed:

- Ability to conduct experiments, as well as to analyze and interpret data (P).
- Ability to use the techniques, skills, and modern engineering tools necessary for engineering practice (P).

Performance	Exceeds expectation (4-5)	Meets expectation (3-2)	Does not meet expectation (1)	Marks
1. Realization of Experiment [a, b]	Selects relevant equipment to the experiment, develops setup diagrams of equipment connections or wiring.	Needs guidance to select relevant equipment to the experiment and to develop equipment connection or wiring diagrams.	Incapable of selecting relevant equipment to conduct the experiment, equipment connection or wiring diagrams are	
2. Conducting Experiment [a, b]	Does proper calibration of equipment, carefully examines equipment moving parts, and ensures smooth operation and process.	Calibrates equipment, examines equipment moving parts, and operates the equipment with minor error.	Unable to calibrate appropriate equipment, and equipment operation is substantially wrong.	
3. Laboratory Safety Rules [a]	Respectfully and carefully observes safety rules and procedures	Observes safety rules and procedures with minor deviation.	Disregards safety rules and procedures.	
6. Data Collection [a]	Plans data collection to achieve experimental objectives, and conducts an orderly and a complete data collection.	Plans data collection to achieve experimental objectives, and collects complete data with minor error.	Does not know how to plan data collection to achieve experimental goals; data collected is incomplete and contain errors.	
7. Data Analysis [a]	Accurately conducts simple computations and statistical analysis using collected data;	Conducts simple computations and statistical analysis using collected data	Unable to conduct simple statistical analysis on collected data; no attempt	

	correlates experimental results to known theoretical values; accounts for measurement errors and parameters that affect experimental results.	with minor error; reasonably correlates experimental results to known theoretical values; attempts to account for measurement errors and parameters that affect experimental results.	to correlate experimental results with known theoretical values; incapable of explaining measurement errors or parameters that affect the experimental results.	
8. Computer Use [a]	Uses computer to collect and analyze data effectively.	Uses computer to collect and analyze data with minor error.	Does not know how to use computer to collect and analyze data.	
Total				

Faculty:

Name:_____

Signature:_____

Date: _____